

Week 1: MLFQ Scheduler Design Document

Multi-Level Feedback Queue Scheduler for xv6-RISC-V

Abdullah Khan Sherwani 27880

Muhammad Anas Tabba 28998

3rd December 2025

Contents

1	Introduction	2
2	MLFQ Design	2
2.1	Queue Architecture	2
2.2	Scheduling Rules	2
3	Implementation	2
3.1	Process Structure (kernel/proc.h)	2
3.2	MLFQ Configuration (kernel/proc.c)	2
3.3	System Call: getprocinfo (kernel/sysproc.c)	3
4	Files Modified	3
5	Testing	4
5.1	procinfo Test	4
5.2	Basic MLFQ Test	4
6	Summary	5

1 Introduction

This document describes the design phase for implementing a Multi-Level Feedback Queue (MLFQ) scheduler in xv6-RISC-V, replacing the default round-robin scheduler with a priority-based system that dynamically adjusts process priorities based on CPU usage behavior.

Week 1 Goals: Design MLFQ queue structure, add data structures to process control block, implement `getprocinfo` system call, and prepare test infrastructure.

2 MLFQ Design

2.1 Queue Architecture

Queue	Priority	Time Quantum	Description
Q0	Highest	2 ticks	Interactive/I/O-bound
Q1	High	4 ticks	Medium-priority
Q2	Medium	8 ticks	Compute-intensive
Q3	Lowest	16 ticks	Long-running CPU-bound

Table 1: MLFQ Queue Configuration

2.2 Scheduling Rules

1. New processes start at Q0 (highest priority)
2. Round-robin within each queue; scheduler picks from highest non-empty queue
3. Process using full time quantum gets demoted; voluntary yield keeps priority
4. **Starvation Prevention:** Every 30 ticks, all processes boost to Q0

3 Implementation

3.1 Process Structure (kernel/proc.h)

```

1 // MLFQ scheduler fields added to struct proc
2 int priority;           // Current queue (0-3, 0 is highest)
3 int time_slices;       // Slices used in current priority
4 uint64 arrival_time;

```

3.2 MLFQ Configuration (kernel/proc.c)

```

1 #define NMLFQ 4
2 #define BOOST_INTERVAL 30
3 int mlfq_time_quantum[NMLFQ] = {2, 4, 8, 16};

```

3.3 System Call: getprocinfo (kernel/sysproc.c)

```
1 uint64 sys_getprocinfo(void) {
2     uint64 addr;
3     struct proc *p = myproc();
4     argaddr(0, &addr);
5
6     struct { int pid, state, priority, time_slices; } info;
7     acquire(&p->lock);
8     info.pid = p->pid;
9     info.state = p->state;
10    info.priority = p->priority;
11    info.time_slices = p->time_slices;
12    release(&p->lock);
13
14    if(copyout(p->pagetable, addr, (char *)&info, sizeof(info)) <
15        0)
16        return -1;
17    return 0;
18 }
```

4 Files Modified

- kernel/proc.h – Added MLFQ fields to struct proc
- kernel/proc.c – MLFQ globals and initialization
- kernel/syscall.h, syscall.c – Registered getprocinfo (syscall 22)
- kernel/sysproc.c – Implemented sys_getprocinfo()
- user/user.h, usys.pl – User-space interface
- Makefile – Set CPUS := 1

6 Summary

Completed: Designed 4-level MLFQ with time quanta [2,4,8,16], added MLFQ fields to PCB, implemented getprocinfo syscall, configured single-CPU mode.

Next (Week 2): Implement MLFQ scheduling logic, time slice tracking, and automatic demotion.